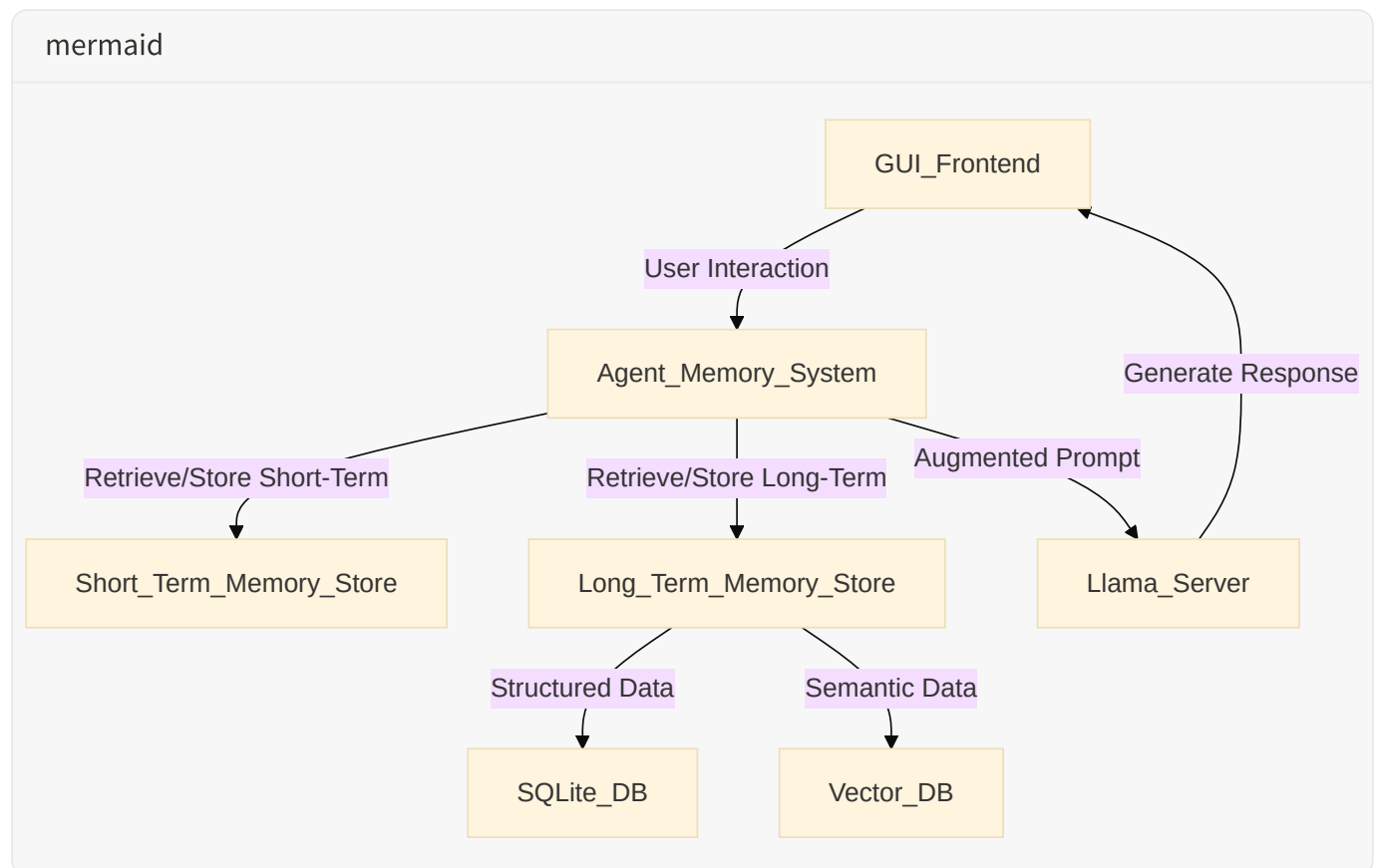


Technical Specification: Agent Memory Implementation

This document outlines the technical specifications for integrating a robust Agent Memory system into the `llama.cpp` GUI wrapper for Fedora KDE Plasma. The goal is to enable the LLM agent to maintain context, learn from interactions, and provide more personalized and consistent responses over time. The implementation will leverage a layered memory architecture, combining transient and persistent storage mechanisms.

1. System Architecture

The Agent Memory system will be a dedicated module within the application, interacting with both the GUI frontend and the `llama-server`. It will employ a layered approach to memory management, as recommended by current best practices [10] [11].



1.1. Components

- **GUI Frontend:** Provides the interface for user interaction, displays LLM responses, and potentially allows users to review or manage their agent's memories.
- **Agent Memory System (Backend Module):** Orchestrates memory operations, including storing new memories, retrieving relevant past interactions, and integrating

them into the LLM's context. It manages the lifecycle of both short-term and long-term memories.

- **Short-Term Memory Store:** A transient storage mechanism, primarily in-memory, for holding recent conversational turns and session-specific context. This is typically managed within the application's runtime environment.
- **Long-Term Memory Store:** A persistent storage layer for enduring knowledge, user preferences, and summarized past interactions. This will be a hybrid system:
 - **SQLite Database:** For structured data such as user preferences, session summaries, and metadata associated with memories [7] [8].
 - **Vector Database:** For semantic search over unstructured memories (e.g., key facts, learned insights) [9] [11]. This could be the same vector database used for RAG or a separate instance.
- **llama-server** : The core LLM inference engine, which receives prompts augmented with retrieved memories from the Agent Memory System.

1.2. Optimization for Local Consumer Hardware

To ensure optimal performance on typical consumer hardware (e.g., AMD CPU 7600X, NVIDIA RTX 4070 GPU 12GB VRAM, 32GB DDR5 RAM), the Agent Memory implementation will adhere to the following principles:

- **Efficient Data Structures:** Utilize highly optimized data structures for in-memory caching and retrieval to minimize CPU overhead and memory footprint.
- **GPU Acceleration for Embeddings:** Leverage the NVIDIA RTX 4070 GPU for accelerating the generation of vector embeddings, ensuring fast processing of new memories and semantic search queries. This will involve using `llama.cpp`'s GPU-accelerated capabilities or dedicated libraries that can utilize the GPU effectively.
- **Minimize Disk I/O:** Prioritize in-memory operations where feasible and optimize disk I/O for persistent storage (SQLite, Vector Database) to reduce latency. Batching writes and reads will be employed.
- **Memory Footprint Management:** Implement strategies to keep the memory footprint low, especially for the short-term memory store, to avoid excessive RAM usage and ensure smooth operation alongside the `llama-server`.
- **Asynchronous Operations:** Handle memory-intensive tasks (e.g., embedding generation, database writes) asynchronously to prevent blocking the main GUI thread and maintain application responsiveness.
- **Configurable Resource Usage:** Allow users to configure resource limits (e.g., cache size, number of CPU threads for certain operations) to fine-tune performance based on

their specific hardware and preferences.

2. Data Models

Well-defined data models are crucial for organizing and retrieving agent memories effectively. The following models will be used for both short-term and long-term memory storage.

2.1. Memory Object Model

This model represents a single piece of information or an event that the agent remembers. It will be stored in both short-term and long-term memory, with long-term memories potentially having additional metadata and vector embeddings.

Field	Type	Description	Example
id	UUID	Unique identifier for the memory object.	mem_id_12345
content	String	The textual content of the memory (e.g., a user statement, an LLM response, a learned fact).	"User prefers concise summaries."
type	Enum	Categorization of the memory (e.g., user_input , llm_output , fact , preference , summary).	preference
importance	Float	A numerical score indicating the perceived importance or salience of the memory.	0.85
timestamp	DateTime	When the memory was created or last updated.	2026-07-04T11:00:00Z
session_id	UUID	Reference to the session in which this memory occurred.	sess_id_abc

metadata	JSON	Arbitrary key-value pairs for additional context (e.g., topic, sentiment).	<code>{"topic": "summarization", "sentiment": "positive"}</code>
embedding	Array<Float>	Vector representation of the <code>content</code> (for long-term semantic search).	<code>[0.2, -0.1, 0.7, ..., 0.3]</code>

2.2. Session Model

This model tracks individual user sessions, providing a container for short-term memories and a mechanism for summarizing interactions for long-term storage.

Field	Type	Description	Example
<code>id</code>	UUID	Unique identifier for the session.	<code>sess_id_abc</code>
<code>user_id</code>	UUID	Identifier for the user associated with the session.	<code>user_id_xyz</code>
<code>start_time</code>	DateTime	Timestamp when the session began.	<code>2026-07-04T10:55:00Z</code>
<code>end_time</code>	DateTime	Timestamp when the session ended (null if active).	<code>null</code>
<code>summary</code>	String	A periodically updated summary of the session's key points (for long-term memory).	<code>"Discussed RAG implementation details."</code>
<code>status</code>	Enum	Current status of the session (e.g., <code>active</code> , <code>ended</code>).	<code>active</code>

2.3. User Profile Model (Optional, for personalization)

This model stores persistent user preferences and information that influences agent

behavior.

Field	Type	Description	Example
id	UUID	Unique identifier for the user.	user_id_xyz
preferences	JSON	User-specific settings (e.g., preferred LLM parameters, output style).	{"verbosity": "high", "tone": "formal"}
created_at	DateTime	Timestamp when the user profile was created.	2026-01-15T09:00:00Z

3. API Endpoints and Interaction Flows

The Agent Memory system will expose a set of RESTful API endpoints to manage the memory lifecycle and facilitate interaction with the GUI frontend and the llama-server . These endpoints will be designed for efficient local operation and seamless integration.

3.1. Memory Management API

This API will handle the creation, retrieval, updating, and deletion of individual memory objects.

Endpoint	Method	Description	Request Body (Example)	Response Body (Example)
/memories	POST	Creates a new memory object.	{ "content": "User prefers concise summaries.", "type": "preference", "session_id": "sess_id_abc" }	{ "id": "mem_id_12345", "status": "created" }
/memories/{id}	GET	Retrieves a specific memory object by its ID.	(None)	{ "id": "mem_id_12345", "content": "..." }
/memories/{id}	PUT	Updates an existing memory object.	{ "importance": 0.9 }	{ "id": "mem_id_12345", "status": "updated" }

/memories/{id}	DELETE	Deletes a specific memory object.	(None)	{"status": "deleted"}
----------------	--------	-----------------------------------	--------	-----------------------

3.2. Session Management API

This API will manage the lifecycle of user sessions and their associated short-term memories.

Endpoint	Method	Description	Request Body (Example)	Response Body (Example)
/sessions	POST	Creates a new user session.	{"user_id": "user_id_xyz"}	{"id": "sess_id_abc", "status": "active"}
/sessions/{id}	GET	Retrieves a specific session and its short-term memories.	(None)	{"id": "sess_id_abc", "memories": [...]}
/sessions/{id}/end	POST	Ends a session and triggers summarization for long-term memory.	(None)	{"id": "sess_id_abc", "status": "ended"}

3.3. Semantic Search API (for Long-Term Memory)

This API will allow the agent to semantically search its long-term memory.

Endpoint	Method	Description	Request Body (Example)	Response Body (Example)
/memory/search	POST	Performs a semantic search across long-term memories.	{"query": "user preferences for summaries", "top_k": 3}	{"relevant_memories": [{"id": "mem_id_1", "content": "...", ...}]}

3.4. Interaction Flow for Memory Retrieval and Update

1. **User Input:** The user interacts with the GUI, generating new input.

2. **GUI to Agent Memory System:** The GUI sends the user input to the Agent Memory System, which stores it as a new short-term memory via `/memories` POST.
3. **Memory Retrieval:** Before generating a response, the Agent Memory System performs several retrieval steps:
 - **Short-Term Context:** Retrieves recent conversational turns from the current session.
 - **Long-Term Semantic Search:** If the query requires broader context or user preferences, it calls `/memory/search` with a relevant query (e.g., based on current user input or session summary) to retrieve relevant long-term memories.
 - **User Profile:** Retrieves user-specific preferences from the User Profile Model.
4. **Prompt Augmentation:** The retrieved short-term context, relevant long-term memories, and user preferences are combined to create an augmented prompt.
5. **Agent Memory System to llama-server :** The augmented prompt is sent to the llama-server .
6. **llama-server Response:** The llama-server generates a response based on the enriched context.
7. **Response Processing & Memory Update:** The Agent Memory System processes the llama-server 's response, potentially summarizing it or extracting key facts to be stored as new long-term memories (via `/memories` POST) or updating the session summary.
8. **llama-server to GUI:** The llama-server sends its response back to the GUI frontend.
9. **GUI Display:** The GUI displays the LLM's response to the user.

References

1. [REST API for AI Models Explained: 2026 Guide](#)
2. [API design best practices guide \(March 2026\)](#)
3. [AI Agent Memory Architecture Guide: Three Paths to the Right ...](#)
4. [AI Agent Memory Systems Compared: RAG vs Local SQLite vs ...](#)
5. [GitHub - sqlalchemy/sqlite-memory: Markdown based AI agent memory ...](#)
6. [How to Build and Operate AI Agent Memory in 2026](#)
7. [I built "SQLite for AI Agents" A local-first memory engine with hybrid ...](#)
8. [A Practical Guide to Memory for Autonomous LLM Agents](#)
9. [How to build your own long-term Agentic Memory System for LLMs](#)